

链式工具调用框架

实验六：链式工具调用框架

一、实验目的

1. 理解链式工具调用的概念和应用场景
2. 掌握多步任务的自动化执行机制
3. 实现基于 JSON 格式的决策交互协议
4. 构建状态管理系统支持复杂任务流程

二、实验内容

2.1 功能概述

本实验实现了一个链式工具调用框架，支持前一个工具的输出作为后一个工具的输入，实现复杂任务的自动化完成。

2.2 核心组件设计

2.2.1 状态管理类 `ChainedCallContext` 类负责管理链式调用的状态：- **steps**: 记录已执行的步骤列表 - **variables**: 存储中间变量供后续步骤使用 - **current_iteration**: 当前迭代次数 - **max_iterations**: 最大迭代限制（默认 10 次）

2.2.2 工具调用协议 定义标准化的 JSON 输出格式：

```
{"done":true,"answer":"任务完成结果"}  
{"done":false,"tool_call":{"name":"read_file","arguments":{"dir_path":"demo","file_name":"1.txt"}}
```

2.2.3 系统提示词 `get_chained_system_prompt()` 定义工具调用规则：- 必须通过调用工具完成任务 - 写入文件必须使用 `create_file` 工具 - 输出格式仅支持 JSON

[迭代 1/10]

决定调用工具: `list_directory`

参数: `{"dir_path": "demo"}`

[迭代 2/10]

决定调用工具: `read_file`

参数: `{"dir_path": "demo", "file_name": "1.txt"}`

[迭代 3/10]

决定调用工具: `read_file`

参数: `{"dir_path": "demo", "file_name": "2.txt"}`

[迭代 4/10]

决定调用工具: `create_file`

参数: `{"dir_path": "demo", "file_name": "result.txt", "content": "42"}`

[完成] 任务完成

四、实验总结

4.1 技术要点

1. 状态持久化: 通过 `ChainedCallContext` 维护跨迭代状态
2. 协议标准化: 定义统一的 JSON 格式协议
3. 容错机制: 实现 JSON 解析的错误恢复和格式修复

4.2 功能扩展

本实验实现了复杂任务的自动化执行能力: - 支持多步骤任务编排 - 工具输出可作为后续输入 - 内置任务完成验证机制

4.3 应用场景

链式工具调用适用于: - 数据处理流水线 - 自动化报告生成 - 多步骤数据分析- 复杂文件操作任务

4.4 改进方向

1. 增加任务规划能力, 自动生成执行计划
2. 实现工具依赖关系分析
3. 添加回滚机制支持任务失败恢复